

Politechnika Rzeszowska im. Ignacego Łukasiewicza

Wydział Matematyki i Fizyki Stosowanej



WYDZIAŁ
MATEMATYKI
I FIZYKI STOSOWANEJ
POLITECHNIKI RZESZOWSKIEJ

System zarządzania agencją mieszkaniową

Mateusz Gałda, Paweł Górski

FSO-DI

Rzeszów 2026

Spis treści

Wstęp i Cel projektu.....	3
Architektura Aplikacji	3
Technologie i struktura	3
Folder Backend	3
Folder Frontend	3
Baza danych	3
Formularz logowania.....	3
Funkcja wyloguj.....	7
Formularz rejestracji	8
System logowania i komunikacja zwrotna.....	9
Implementacja modułu rejestracji użytkowników	15
Implementacja bezpieczeństwa	17
Ochrona przed SQL inject.....	17
Ochrona przed CRSF	18
Bezpieczeństwo uwierzytelniania i Sesje.....	19
Zarządzanie Sesją	19
Walidacja danych	19
Analiza poprawności kodu HTML strony internetowej	20
Testy penetracyjne za pomocą aplikacji ZAP	21
Nowoczesna ochrona przed botami – Cloudflare Turnstile	22
Charakterystyka technologii.....	22
Uzasadnienie wyboru	22
Szczegółowy opis interfejsu użytkownika i nawigacji.....	23

Wstęp i Cel projektu

Celem projektu było stworzenie działającej aplikacji internetowej, następnie zapewnienie jej zabezpieczeń. Projekt przedstawia działającą witrynę agencji nieruchomości w architekturze klient-serwer, z wykorzystaniem technologii PHP, MySQL oraz JS.

Architektura Aplikacji

Technologie i struktura

Folder Backend

Zawiera pliki kluczowe dla logiki strony zapewniające jej biznesowe działanie. Są tam takie pliki jak: api.php, auth.php oraz config.php.

Folder Frontend

Zawiera pliki odpowiedzialne za wizualną prezentację strony między innymi app.js do dynamicznego pobierania danych z bazy, następnie style.css odpowiadające za oprawę wizualną.

Baza danych

Dodatkowo jest zamieszczony plik backup bazy danych, która jest połączona z witryną.

Formularz logowania

Jeżeli podane dane będą błędne pojawia się komunikat o podaniu błędnych danych oraz ostrzeżenie o pozostałych próbach.

Zaloguj się

Email
mail@mail.com

Hasło
Hasło

Powodzenie! Przestrzeń Własności

Zaloguj się

✗ Błędny email lub hasło Pozostałe próby: 2/3

Nie masz konta? [Zarejestruj się](#)

[Powrót do strony głównej](#)

Rys. 1 Formularz logowania po podaniu błędnych danych

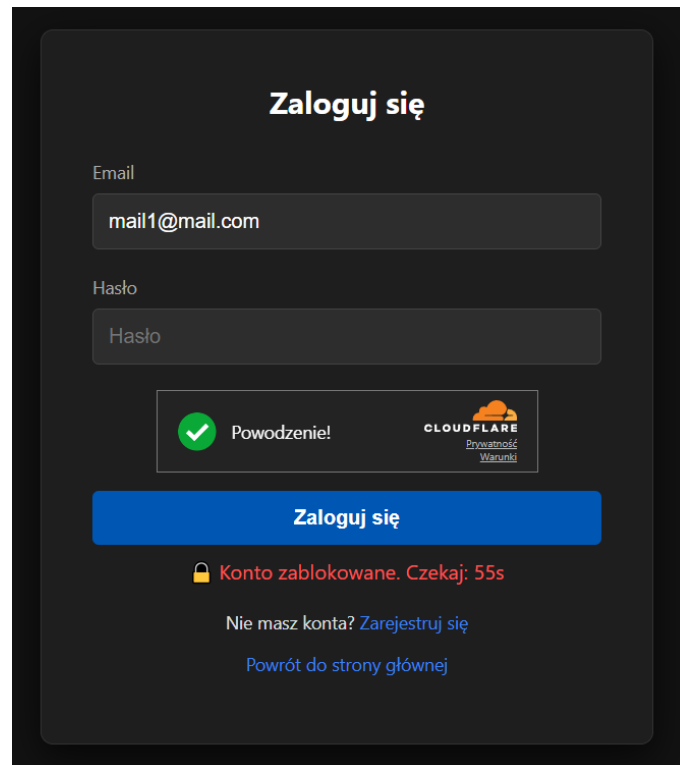
```
// BŁĄD LOGOWANIA - zwiększ licznik
lockData.attempts = (lockData.attempts || 0) + 1;
const attemptsLeft = 3 - lockData.attempts;

// Odliczanie
let secondsLeft = 60;
const blockInterval = setInterval(() => {
  secondsLeft--;
  messageBox.innerText = `🔒 Konto zablokowane. Czekaj: ${secondsLeft}s`;

  if (secondsLeft <= 0) {
    clearInterval(blockInterval);
    messageBox.innerText = '✅ Możesz spróbować zalogować się ponownie.';
    messageBox.style.color = '#4CAF50';
    submitBtn.disabled = false;
  }
}
```

Powyższy kod przedstawia logikę zliczanie błędów logowania oraz mechanizm nakładania blokady czasowej na 60s gdy będą 3 błędy.

W sytuacji kiedy podamy 3 razy błędne dane zostanie nam odebrany dostęp do tego formularza na 1 minutę:



Rys. 2 Formularz logowania z odebraniem dostępu

```
if (lockData.blockedUntil && now < lockData.blockedUntil) {  
    const secondsLeft = Math.ceil((lockData.blockedUntil - now) / 1000);  
    messageBox.innerText = `❌ Konto zablokowane na ${secondsLeft} sekund;  
    messageBox.style.color = '#ff5555';  
    submitBtn.disabled = true;  
  
    const countdownInterval = setInterval(() => {  
        const remaining = Math.ceil((lockData.blockedUntil - Date.now()) / 1000);  
        if (remaining <= 0) {  
            clearInterval(countdownInterval);  
            messageBox.innerText = "";  
            submitBtn.disabled = false;  
            localStorage.removeItem(lockKey);  
        } else {  
            messageBox.innerText = `❌ Konto zablokowane na ${remaining} sekund. Spróbuj później.`;  
        }  
    }, 1000);
```

```

    return;
}

// BŁĄD LOGOWANIA - zwiększ licznik
    lockData.attempts = (lockData.attempts || 0) + 1;
    const attemptsLeft = 3 - lockData.attempts;

    if (lockData.attempts >= 3) {
        // BLOKADA NA 60 SEKUND
        lockData.blockedUntil = now + 60000;
        lockData.attempts = 0;
        localStorage.setItem(lockKey, JSON.stringify(lockData));

        messageBox.innerText = '🔒 Zbyt wiele błędnych prób! Konto zablokowane na 60
sekund.';

        messageBox.style.color = '#ff5555';
        submitBtn.disabled = true;
        passwordInput.value = "";

        // Odliczanie
        let secondsLeft = 60;
        const blockInterval = setInterval(() => {
            secondsLeft--;
            messageBox.innerText = `🔒 Konto zablokowane. Czekaj: ${secondsLeft}s`;

            if (secondsLeft <= 0) {
                clearInterval(blockInterval);
                messageBox.innerText = '✅ Możesz spróbować zalogować się ponownie.';
                messageBox.style.color = '#4CAF50';
                submitBtn.disabled = false;
            }
        }, 1000);
    }

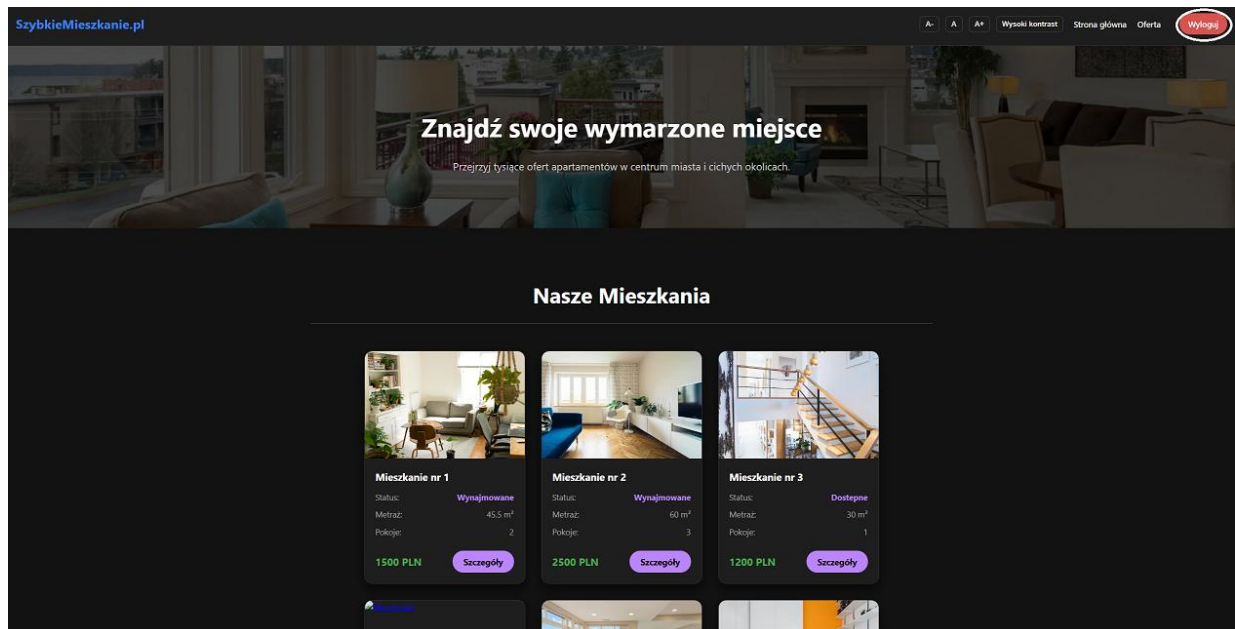
```

Powyższy kod przedstawia logikę działania wyświetlania pozostałych prób oraz zablokowania formularza na minutę.

Funkcja wyloguj

- System wylogowania

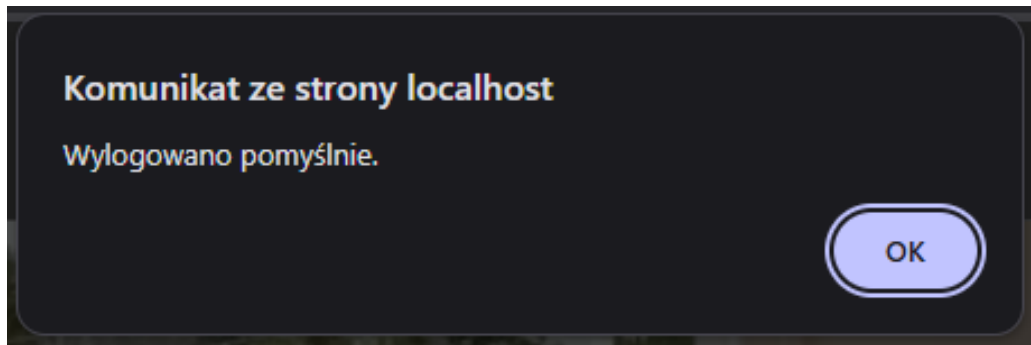
Po prawej stronie nagłówka po zalogowaniu przycisk zaloguj się, zostaje zmieniony na wyloguj:



Rys. 1 Strona główna z zaznaczonym przyciskiem wyloguj

```
if (navLogoutBtn) {  
  
    navLogoutBtn.addEventListener('click', function(e) {  
        e.preventDefault();  
  
        localStorage.removeItem('userLoggedIn');  
        alert("Wylogowano pomyślnie.");  
  
        window.location.reload();  
    });  
}
```

Po przyciśnięciu otrzymujemy komunikat ze strony o wylogowaniu, a następnie jest kasowana nasza sesja i tracimy dostęp do informacji na stronie:



Rys. 2 Komunikat ze strony przy wylogowywaniu

```
// Obsługa kliknięcia "Wyloguj"
if (navLogoutBtn) {
  navLogoutBtn.addEventListener('click', function(e) {
    e.preventDefault();

    localStorage.removeItem('userLoggedIn');
    alert("Wylogowano pomyślnie.");

    window.location.reload();
  });
}
```

Powyższy kod przedstawia działania komunikatu wylogowania.

Formularz rejestracji

- System rejestracji

Formularz logowania zawiera 4 pola: Imię, nazwisko, adres mail oraz hasło, należy je uzupełnić poprawnymi danymi aby konto zostało utworzone

The image shows a registration form titled "Zarejestruj się" (Register) on a dark background. The form contains four input fields: "Imię" (Name) with the value "Jan", "Nazwisko" (Surname) with the value "Kowalski", "Email" with the value "twoj@email.com", and "Hasło" (Password) with the placeholder "Hasło (min. 6 znaków)". Below the fields is a green checkmark icon and the text "Powodzenie!" (Success!). To the right of this is the Cloudflare logo and the text "Chroń swoją stronę" (Protect your site). At the bottom of the form is a blue button labeled "Zarejestruj się". Below the button are two links: "Masz już konto? Zaloguj się" (Already have an account? Log in) and "Powrót do strony głównej" (Return to the main page).

Rys. 3 Formularz logowania

System logowania i komunikacja zwrotna

Interfejs logowania (Rys. 1) jest ściśle zintegrowany z mechanizmami bezpieczeństwa opisanymi w sekcji technicznej.

- **Obsługa błędów:** W przypadku podania nieprawidłowych danych, system nie tylko informuje o błędzie, ale również dynamicznie wylicza i wyświetla liczbę pozostałych prób (np. "Pozostałe próby: 2/3").
- **Blokada czasowa:** Po przekroczeniu limitu trzech błędnych prób, interfejs przechodzi w stan blokady. Przycisk "Zaloguj się" zostaje dezaktywowany, a użytkownik widzi czerwony komunikat z odliczaniem sekund pozostałych do odblokowania formularza (np. "Konto zablokowane. Czekaj: 55s").
- **Potwierdzenie wylogowania:** Po wybraniu opcji wylogowania, aplikacja wyświetla systemowe powiadomienie "Wylogowano pomyślnie", a następnie odświeża stronę, czyszcząc dane sesji z pamięci lokalnej przeglądarki.

```
// REJESTRACJA
```

```
const registerForm = document.getElementById('registerForm');
```

```
if (registerForm) {
```

```
  registerForm.addEventListener('submit', function(e) {
```

```
    e.preventDefault();
```

```
    const imieInput = document.getElementById('imie');
```

```
    const nazwiskoInput = document.getElementById('nazwisko');
```

```
    const emailInput = document.getElementById('reg-email');
```

```
    const passwordInput = document.getElementById('reg-password');
```

```
    const regMessageBox = document.getElementById('reg-message');
```

```
    const submitBtn = registerForm.querySelector('button[type="submit"]');
```

```
    if (!imieInput || !nazwiskoInput || !emailInput || !passwordInput) {
```

```
      alert("Błąd: Formularz rejestracji nie ma wymaganych pól!");
```

```
      return;
```

```
    }
```

```
    const imieVal = imieInput.value.trim();
```

```
    const nazwiskoVal = nazwiskoInput.value.trim();
```

```
    const emailVal = emailInput.value.trim();
```

```
    const passVal = passwordInput.value;
```

```
    if (!imieVal || !nazwiskoVal || !emailVal || !passVal) {
```

```
      regMessageBox.innerText = '✗ Wypełnij wszystkie pola!';
```

```
      regMessageBox.style.color = '#ff5555';
```

```
      return;
```

```
    }
```

```
    if (!isValidEmail(emailVal)) {
```

```
      regMessageBox.innerText = '✗ Wpisz poprawny email!';
```

```
      regMessageBox.style.color = '#ff5555';
```

```
      return;
```

```
    }
```

```
if (passVal.length < 6) {  
    regMessageBox.innerText = '✗ Hasło musi mieć co najmniej 6 znaków!';  
    regMessageBox.style.color = '#ff5555';  
    return;  
}
```

```
regMessageBox.innerText = 'Trwa rejestracja...';  
regMessageBox.style.color = '#666';  
submitBtn.disabled = true;
```

```
const formData = new FormData(registerForm);  
const turnstileResponse = formData.get('cf-turnstile-response');  
fetch('../backend/api.php?action=register', {  
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify({  
        imie: imieVal,  
        nazwisko: nazwiskoVal,  
        email: emailVal,  
        password: passVal,  
        'cf-turnstile-response': turnstileResponse  
    })  
})  
    .then(response => response.json())  
    .then(data => {  
        if (data.status === 'success') {  
            regMessageBox.innerText = '☑ ' + data.message;  
            regMessageBox.style.color = '#4CAF50';  
            registerForm.reset();  
  
            setTimeout(() => {  
                if (switchToLogin) switchToLogin.click();  
            }, 2000);  
        } else {  
            regMessageBox.innerText = '✗ ' + data.message;  
            regMessageBox.style.color = '#ff5555';  
        }  
    })
```

```

        submitBtn.disabled = false;
    })
    .catch(error => {
        console.error('Błąd:', error);
        regMessageBox.innerText = '⚠ Błąd połączenia z serwerem!';
        regMessageBox.style.color = '#ff5555';
        submitBtn.disabled = false;
    });
});
}

```

Przy podaniu w polu Email czegoś co nim nie jest, strona zwróci następujący komunikat:

The screenshot shows a registration form titled "Zarejestruj się" (Register) with the following fields and content:

- Imię** (First Name): Input field containing "test".
- Nazwisko** (Last Name): Input field containing "malla".
- Email**: Input field containing "123123123123".

A validation error message is displayed below the email field:

⚠ Uwzględnij znak „@” w adresie e-mail. W adresie „123123123123” brakuje znaku „@”.

Below the error message is a password field with masked characters ".....".

At the bottom of the form, there is a green checkmark icon and the text "Powodzenie!" (Success!). To the right of this is the Cloudflare logo and the text "Prywatność Warunki" (Privacy Policy Terms).

Below the success message is a blue button labeled "Zarejestruj się" (Register).

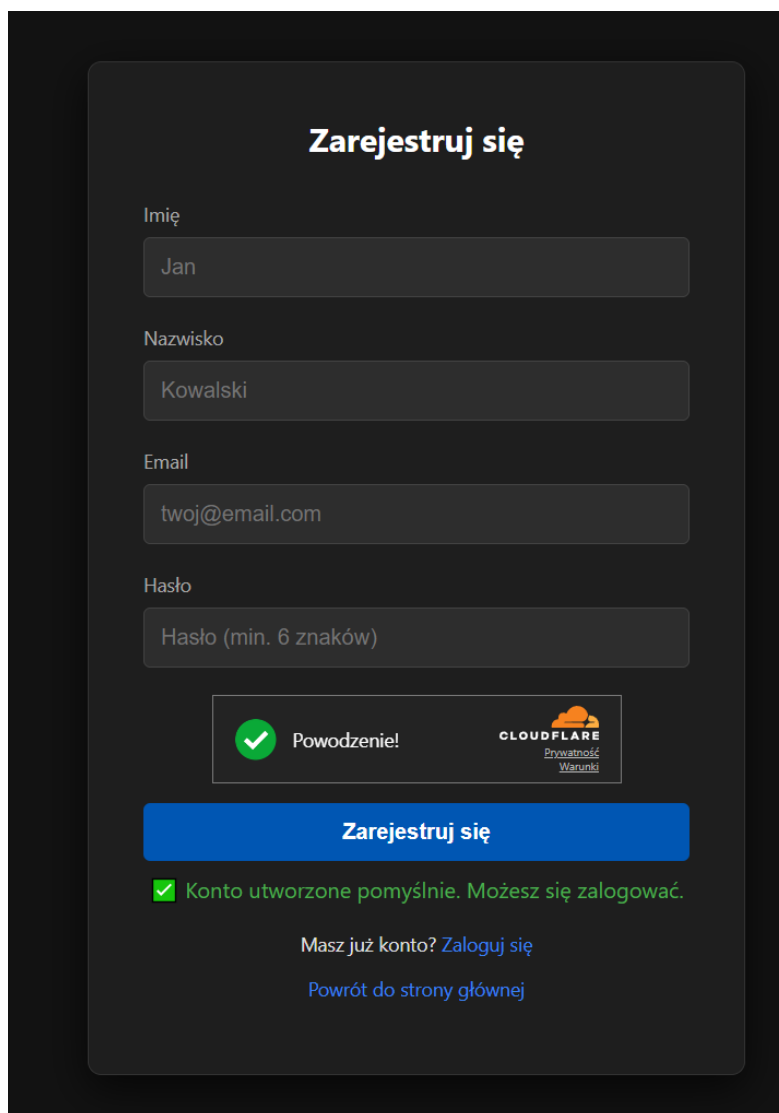
At the very bottom, there are two links: "Masz już konto? Zaloguj się" (Already have an account? Log in) and "Powrót do strony głównej" (Return to the main page).

Rys. 4 Komunikat strony na fałszywy mail

Wynika to z walidacji podawanych danych w formularzu:

```
function isValidEmail(email) {
    const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    return regex.test(email);
}
if (!isValidEmail(emailVal)) {
    regMessageBox.innerText = '✗ Wpisz poprawny email!';
    regMessageBox.style.color = '#ff5555';
    return;
}
```

Po podaniu poprawnych danych konto zostaje zapisane w bazie danych z zaszyfrowanym hasłem:



The image shows a registration form titled "Zarejestruj się" (Register) on a dark background. The form contains four input fields: "Imię" (Name) with the value "Jan", "Nazwisko" (Surname) with the value "Kowalski", "Email" with the value "twoj@email.com", and "Hasło" (Password) with the placeholder "Hasło (min. 6 znaków)". Below the fields is a success message box with a green checkmark icon, the text "Powodzenie!" (Success!), and the Cloudflare logo with the text "Prywatność" (Privacy) and "Warunki" (Terms). Below the success message is a blue button labeled "Zarejestruj się". At the bottom, there is a green checkmark icon followed by the text "Konto utworzone pomyślnie. Możesz się zalogować." (Account created successfully. You can log in.), and two links: "Masz już konto? Zaloguj się" (Already have an account? Log in) and "Powrót do strony głównej" (Return to the main page).

Rys. 5 Komunikat strony na pomyślną rejestrację konta

```

const turnstileResponse = formData.get('cf-turnstile-response');

fetch('../backend/api.php?action=register', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    imie: imieVal,
    nazwisko: nazwiskoVal,
    email: emailVal,
    password: passVal,
    'cf-turnstile-response': turnstileResponse
  })
})
.then(response => response.json())
.then(data => {
  if (data.status === 'success') {
    regMessageBox.innerText = '✅ ' + data.message;
    regMessageBox.style.color = '#4CAF50';
    registerForm.reset();

    setTimeout(() => {
      if (switchToLogin) switchToLogin.click();
    }, 2000);
  } else {
    regMessageBox.innerText = '❌ ' + data.message;
    regMessageBox.style.color = '#ff5555';
  }
  submitBtn.disabled = false;
})
.catch(error => {
  console.error('Błąd:', error);
  regMessageBox.innerText = '⚠️ Błąd połączenia z serwerem!';
  regMessageBox.style.color = '#ff5555';
  submitBtn.disabled = false;
});
});

```

Powyższy kod przedstawia działanie komunikatów formularza rejestracji.

Implementacja modułu rejestracji użytkowników

Moduł rejestracji jest kluczowym elementem systemu, odpowiedzialnym za bezpieczne wprowadzanie nowych użytkowników do bazy danych agencji. Proces ten został zaprojektowany z uwzględnieniem wielopoziomowej walidacji danych oraz mechanizmów ochrony przed automatycznymi skryptami.

1. Struktura formularza i gromadzone dane

Interfejs rejestracji (Rys. 11) został ograniczony do niezbędnego minimum danych, co optymalizuje proces zakładania konta. Formularz składa się z czterech głównych pól wejściowych:

- **Imię oraz Nazwisko:** Pola tekstowe służące do identyfikacji personalnej klienta.
- **Adres Email:** Pełni rolę unikalnego identyfikatora (loginu) w systemie.
- **Hasło:** Pole typu *password*, zamaskowane w celu ochrony prywatności podczas wpisywania.

Dodatkowo, w strukturze formularza zaimplementowano ukryty kontener dla mechanizmu Cloudflare Turnstile, który generuje token weryfikacyjny przesyłany wraz z danymi użytkownika.

2. Mechanizmy walidacji danych

System wykorzystuje rygorystyczne reguły sprawdzania poprawności wprowadzonych informacji jeszcze przed wysłaniem żądania do serwera:

- **Weryfikacja kompletności:** Skrypt sprawdza, czy żadne z pól nie pozostało puste. W przypadku braku danych wyświetlany jest komunikat: „Wypełnij wszystkie pola!”.
- **Walidacja formatu Email:** Zastosowano wyrażenie regularne `(/^[^s@]+@[^s@]+\.[^s@]+$/)`, które weryfikuje, czy wprowadzony ciąg znaków posiada strukturę poprawną dla adresu poczty elektronicznej. Próba użycia błędnego formatu skutkuje wyświetleniem błędu: „Wpisz poprawny email!”.
- **Sila hasła:** System wymaga, aby hasło składało się z co najmniej 6 znaków. Jest to podstawowe zabezpieczenie przed wybieraniem bardzo krótkich, łatwych do odgadnięcia haseł.

3. Proces przetwarzania i komunikacja z API

Po pomyślnej walidacji po stronie klienta, dane są pakowane do formatu JSON i przesyłane metodą POST do skryptu backendowego `api.php?action=register`.

- **Asynchroniczność:** Wykorzystanie interfejsu fetch pozwala na rejestrację bez przeładowywania całej strony, co znacząco poprawia komfort użytkowania (UX).
- **Zabezpieczenie hasła:** Dane nie są zapisywane w bazie w formie jawnej. Hasło poddawane jest procesowi haszowania po stronie serwera, co sprawia, że nawet w przypadku wycieku bazy danych, pozostaje ono niemożliwe do odczytania.
- **Zapis w bazie:** Informacje o nowym kliencie trafiają do tabeli klient, gdzie automatycznie nadawany jest unikalny identyfikator id_klienta oraz rejestrowana jest data dołączenia.

4. Komunikaty zwrotne i obsługa błędów

System dynamicznie reaguje na odpowiedź z serwera, informując użytkownika o statusie operacji:

- **Sukces:** Wyświetlany jest zielony komunikat „Konto utworzone pomyślnie. Możesz się zalogować”, a formularz zostaje wyczyszczony. Następnie użytkownik jest automatycznie przekierowywany do widoku logowania.
- **Błąd serwera/duplikacja:** Jeśli adres email istnieje już w bazie lub wystąpił problem z połączeniem, użytkownik otrzymuje stosowne ostrzeżenie (np. „Błąd połączenia z serwerem!”)

[illegible]

Rys. 6 Konta użytkowników ze strony bazy danych

Tabela klient stanowi centralny punkt składowania danych w systemie i została zaprojektowana z uwzględnieniem zasad integralności danych oraz bezpieczeństwa teleinformatycznego. Każdy atrybut w tabeli pełni określoną rolę w procesie autoryzacji i identyfikacji użytkownika. Tabela bezpośrednio wspiera wdrożone w aplikacji warstwy ochronne:

- **Ochrona przed wyciekami danych (Data-at-Rest Protection):** Zastosowanie typu VARCHAR(255) dla kolumny hasło nie jest przypadkowe. Pozwala ono na przechowywanie silnych skrótów generowanych funkcją password_hash(), co sprawia, że w przypadku uzyskania nieuprawnionego dostępu do bazy danych, poświadczenia użytkowników pozostają bezpieczne i niemożliwe do odczytania.
- **Odporność na ataki typu SQL Injection:** Definicje typów danych (np. INT dla identyfikatorów) wymuszają na silniku bazy danych rygorystyczne sprawdzanie wartości. W połączeniu z mechanizmem **Prepared Statements** (zapytania przygotowane), gdzie wartości z pól takich jak email_klienta są przesyłane jako parametry, uniemożliwia to wykonanie złośliwego kodu SQL wewnątrz silnika bazy danych.
- **Zarządzanie sesją i integralność (Referential Integrity):** Wykorzystanie silnika InnoDB (widocznego w konfiguracji tabeli) pozwala na stosowanie kluczy obcych (adres_id_adresu), co gwarantuje, że relacje między danymi są zawsze spójne i uniemożliwia istnienie „osieroconych” rekordów, co mogłoby zostać wykorzystane w atakach logicznych na aplikację.

Implementacja bezpieczeństwa

Ochrona przed SQL inject

Wszystkie zapytania kierowane do strony poprzez api.php czy pobierz_szczegoly.php wykorzystują Prepared Statements, czyli mają przygotowane gotowe zapytania do bazy, co uniemożliwia wstrzyknięcie komend SQL do bazy danych.

```
$pdo = new PDO("mysql:host=$host;dbname=$db_name;charset=utf8", $username, $password);  
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

```
$stmt = $pdo->prepare("SELECT * FROM klient WHERE email_klienta = ?");  
$stmt->execute([$email]);
```

```
$user = $stmt->fetch();
```

Użycie \$email zamiast bezpośredniego wpisywania uniemożliwia wpisanie niebezpiecznego polecenia SQL do prosto do bazy.

Ochrona przed CSRF

Atak Cross-Site Request Forgery (CSRF) polega na wymuszeniu przez atakującego wykonania niechcianej akcji w aplikacji internetowej, w której ofiara jest aktualnie zalogowana i posiada aktywaną sesję. Ochrona realizowana jest w modelu **Synchronizer Token Pattern** z wykorzystaniem komunikacji JSON:

- **Generowanie:** Przy starcie sesji backend (PHP) tworzy unikalny, losowy ciąg znaków i zapisuje go w zmiennej sesyjnej.

```
if (empty($_SESSION['csrf_token'])) {  
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));  
}
```

- **Przekazywanie:** Frontend (JavaScript) pobiera ten token metodą GET przy inicjalizacji strony i przechowuje go w zmiennej.

```
if (empty($input['csrf_token']) || !hash_equals($_SESSION['csrf_token'], $input['csrf_token'])) {  
    echo json_encode(['success' => false, 'message' => 'Błąd bezpieczeństwa (CSRF). Odśwież  
stronę.']);  
    exit;
```

- **Weryfikacja:** Przy każdej próbie logowania lub rejestracji, JavaScript dołącza token do przesyłanych danych JSON. Backend porównuje token otrzymany z tym zapisanym w sesji – jeśli się nie zgadzają, żądanie jest natychmiast odrzucane.

```
if (empty($input['csrf_token']) || !hash_equals($_SESSION['csrf_token'], $input['csrf_token'])) {  
    echo json_encode(['success' => false, 'message' => 'Błąd bezpieczeństwa (CSRF). Odśwież  
stronę.']);  
    exit;
```

Bezpieczeństwo uwierzytelniania i Sesje

Hasła przy rejestracji i logowaniu są haszowane co uniemożliwia ich przejęcie w formie jawnej.

```
if ($user && password_verify($pass, $user['haslo'])) {  
    $_SESSION['user_id'] = $user['id_klienta'];  
    $_SESSION['user_name'] = $user['imie_klienta'];  
  
    echo json_encode([  
        'success' => true,  
        'message' => 'Zalogowano!',  
        'user_name' => $user['imie_klienta'],  
        'redirect' => '../public/index.html'  
    ]);  
} else {  
    echo json_encode(['success' => false, 'message' => 'Błędny email lub hasło']);  
}
```

Funkcja password_verify porównuje wpisany tekst z haszem z bazy danych.

Zarządzanie Sesją

Mechanizm wylogowania w logout.php niszczy sesję po stronie serwera co uniemożliwia przejęcia sesji – session hijacking.

```
<?php  
session_start();  
session_unset();  
session_destroy();  
header('Content-Type: application/json');  
echo json_encode(['success' => true]);  
?>
```

Walidacja danych

Dane podawane przy rejestracji i logowaniu są filtrowane przed przetwarzaniem.

```
$email = $input['email'];  
$pass = $input['password'];
```

Analiza poprawności kodu HTML strony internetowej

Przeprowadzony test został wykonany za pomocą narzędzia **Nu HTML Checker**, które służy do automatycznej walidacji kodu HTML pod kątem zgodności ze specyfikacją HTML5. Narzędzie to analizuje strukturę dokumentu, poprawność składni znaczników oraz wartości atrybutów, a następnie wskazuje błędy i ostrzeżenia, które mogą powodować problemy z poprawnym wyświetlaniem strony w przeglądarkach lub jej interpretacją przez narzędzia zewnętrzne. Sprawdzana została strona dostępna publicznie pod adresem wygenerowanym przez usługę **ngrok**, co oznacza, że test dotyczył realnie działającej wersji strony, a nie jedynie plików lokalnych.

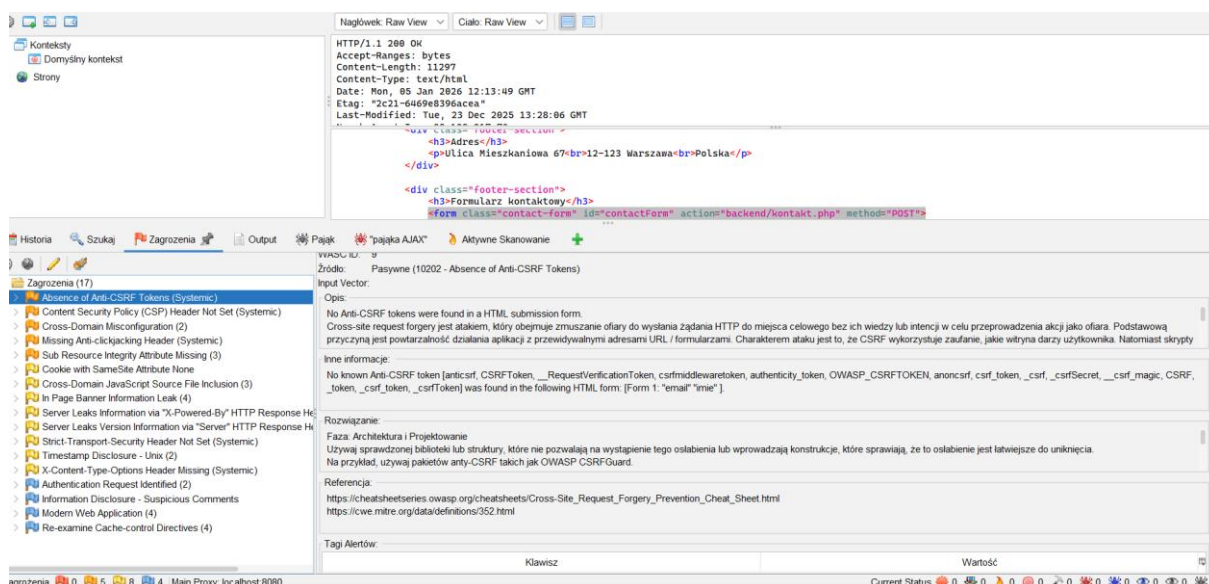
The screenshot shows the Nu HTML Checker interface. At the top, it says "Nu Html Checker" and "This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change". Below that, it shows the URL being checked: "https://a28a90b313f3.ngrok-free.app/agencja1/public/ (checked with vnu 26.1.5)". The "Checker Input" section has a "Show" dropdown set to "address" and a "Check" button. Below the input section, there is a "Message Filtering" section with two error messages:

- Error** Bad value '100%' for attribute 'width' on element 'iframe': Expected a digit but saw '%'.
From line 78, column 21; to line 78, column 334
`<iframe src="https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d2443.837!2d21.012228!3d52.229676!_spl!4v1639999999999" width="100%" height="100%" style="border:0;" allowfullscreen="" loading="lazy"></ifra`
- Error** Bad value '100%' for attribute 'height' on element 'iframe': Expected a digit but saw '%'.
From line 78, column 21; to line 78, column 334
`<iframe src="https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d2443.837!2d21.012228!3d52.229676!_spl!4v1639999999999" width="100%" height="100%" style="border:0;" allowfullscreen="" loading="lazy"></ifra`

Wynik przeprowadzonego testu wskazuje, że strona internetowa została napisana poprawnie i spełnia swoje założenia funkcjonalne. Wykryte błędy mają charakter marginalny i dotyczą wyłącznie formalnej zgodności kodu ze standardem **HTML**. Błąd polega na zastosowaniu wartości procentowych w atrybutach **width** oraz **height** znacznika **iframe**, w którym osadzona jest mapa Google. Zgodnie ze specyfikacją HTML atrybuty te powinny przyjmować jedynie wartości liczbowe, dlatego walidator zgłasza je jako niepoprawne. Jest to jednak nieistotne z punktu widzenia działania strony, ponieważ współczesne przeglądarki poprawnie interpretują takie wartości. Dodatkowo taki sposób zapisu pochodzi bezpośrednio z gotowego kodu udostępnianego przez API Google Maps, co oznacza, że mimo formalnej niezgodności ze standardem HTML rozwiązanie to jest powszechnie stosowane, bezpieczne w użyciu i nie wpływa na poprawne wyświetlanie strony, dlatego błędy te nie są uznawane za krytyczne.

Testy penetracyjne za pomocą aplikacji ZAP

Do przeprowadzenia testów bezpieczeństwa aplikacji wykorzystano narzędzie OWASP ZAP (Zed Attack Proxy), będące jednym z najpopularniejszych na świecie skanerów typu DAST (Dynamic Application Security Testing). Program ten funkcjonuje jako pośrednik typu "Man-in-the-Middle" między przeglądarką a serwerem, co umożliwia przechwytywanie, analizowanie oraz modyfikowanie ruchu sieciowego w czasie rzeczywistym. W ramach badania zastosowano dwuetapowe podejście obejmujące skanowanie pasywne, polegające na analizie nagłówków i odpowiedzi serwera bez ingerencji w działanie systemu, oraz skanowanie aktywne, które symuluje rzeczywiste ataki hakierskie poprzez wysyłanie zmanipulowanych danych do formularzy.



Przeprowadzony audyt wykazał obecność łącznie 17 potencjalnych zagrożeń, z których zdecydowana większość miała charakter informacyjny i dotyczyła brakujących nagłówków bezpieczeństwa w konfiguracji serwera deweloperskiego, co nie stanowi krytycznego błędu w samej logice kodu. Należy odnotować, że mimo poprawnego wdrożenia zabezpieczeń, skaner OWASP ZAP w raporcie końcowym nadal sygnalizuje brak tokenów CSRF. Jest to klasyczny przykład fałszywego alarmu (False Positive), wynikający z ograniczeń automatu, który poszukuje statycznych pól w kodzie HTML i nie potrafi poprawnie zinterpretować dynamicznego przesyłania tokenów w formacie JSON.

Nowoczesna ochrona przed botami – Cloudflare Turnstile

Charakterystyka technologii

W ramach projektu zdecydowano się na rezygnację z tradycyjnych systemów typu **reCAPTCHA** na rzecz rozwiązania **Cloudflare Turnstile**. Jest to inteligentny system weryfikacji użytkownika, który nie wymaga od niego rozwiązywania uciążliwych zagadek wizualnych (np. wskazywania przejść dla pieszych czy sygnalizacji świetlnej). Turnstile wykorzystuje niewidoczne wyzwania przeglądarkowe, analizując zachowanie i sygnały telemetryczne w celu odróżnienia człowieka od zautomatyzowanego skryptu (bota).

Uzasadnienie wyboru

Główne powody wdrożenia tej technologii w systemie agencji mieszkaniowej to:

- **Poprawa UX (User Experience):** Użytkownik nie musi przerywać procesu rejestracji lub logowania, aby udowodnić, że jest człowiekiem.
- **Ochrona prywatności:** W przeciwieństwie do rozwiązań konkurencyjnych, Turnstile nie śledzi użytkowników w celach reklamowych.
- **Skuteczność:** Zapewnia wysoką odporność na ataki typu *credential stuffing* oraz masową rejestrację fałszywych kont.

Implementacja w systemie

Wdrożenie ochrony Turnstile zostało zrealizowane w modelu hybrydowym, obejmującym zarówno warstwę prezentacji (Frontend), jak i logiki biznesowej (Backend):

1. Warstwa Frontendu: W formularzach logowania oraz rejestracji umieszczono dedykowany kontener wizualny. JavaScript dynamicznie pobiera unikalny token odpowiedzi po pomyślnym przejściu niewidocznego testu.

- Token jest pobierany z danych formularza za pomocą metody `formData.get('cf-turnstile-response')`.
- Następnie jest on dołączany do obiektu JSON przesyłanego do serwera.

2. Warstwa Backendowa (API): Serwer po otrzymaniu danych nie procesuje ich od razu. Zanim zapytanie SQL zostanie wykonane, system przesyła otrzymany token do serwerów

Cloudflare w celu weryfikacji jego autentyczności. Dopiero pozytywny wynik weryfikacji pozwala na dalsze przetwarzanie logowania lub rejestracji.

Wpływ na wyniki testów bezpieczeństwa

Zastosowanie Cloudflare Turnstile bezpośrednio przekłada się na bezpieczeństwo formularzy przed atakami typu Brute-force. Nawet jeśli atakujący zna schemat działania API systemu, nie jest w stanie wysłać poprawnego żądania bez unikalnego, jednorazowego tokena wygenerowanego przez silnik Turnstile. Stanowi to dodatkową, zewnętrzną warstwę ochrony, uzupełniającą mechanizmy blokady czasowej zaimplementowane lokalnie w kodzie JavaScript.

Szczegółowy opis interfejsu użytkownika i nawigacji

System Zarządzania Agencją Mieszkaniową został zaprojektowany w architekturze klient-serwer, z naciskiem na przejrzystość prezentowanych ofert oraz bezpieczeństwo procesu uwierzytelniania. Interfejs opiera się na dynamicznej komunikacji z bazą danych, co pozwala na bieżące aktualizowanie stanów widocznych dla użytkownika.

1. Strona główna i nawigacja

Główny panel aplikacji (index.html) stanowi centrum interakcji z użytkownikiem.

- **Nagłówek (Header):** Zawiera logotyp strony "SzybkieMieszkanie.pl" oraz menu nawigacyjne umożliwiające przejście do strony głównej oraz aktualnej oferty.
- **Moduł Dostępności:** W górnej części interfejsu zaimplementowano narzędzia wspomagające dostępność (Accessibility), w tym przyciski do zmiany wielkości czcionki (A-, A, A+) oraz przełącznik wysokiego kontrastu, co jest kluczowe dla użytkowników z dysfunkcjami wzroku.
- **Dynamiczny przycisk sesji:** Logika interfejsu automatycznie zmienia dostępny przycisk w nagłówku – przed zalogowaniem użytkownik widzi opcję „Zaloguj się”, która po pomyślnej autoryzacji zmienia się w funkcję „Wyloguj”.

2. Interfejs Rejestracji i Walidacja

- Formularz rejestracji (Rys. 11) został zaprojektowany tak, aby minimalizować ryzyko wprowadzenia błędnych danych do bazy.
- **Wymagane pola:** Użytkownik musi podać imię, nazwisko, unikalny adres email oraz hasło o długości minimum 6 znaków.

- **Walidacja w czasie rzeczywistym:** System wykorzystuje wyrażenia regularne (Regex) do sprawdzania poprawności formatu adresu email. W przypadku podania błędnego ciągu znaków, interfejs wyświetla dymek ostrzegawczy z prośbą o uwzględnienie znaku "@".
- **Zabezpieczenie Turnstile:** Formularz jest chroniony przez widget Cloudflare Turnstile, który wymaga pomyślnej weryfikacji przed wysłaniem danych do serwera.